

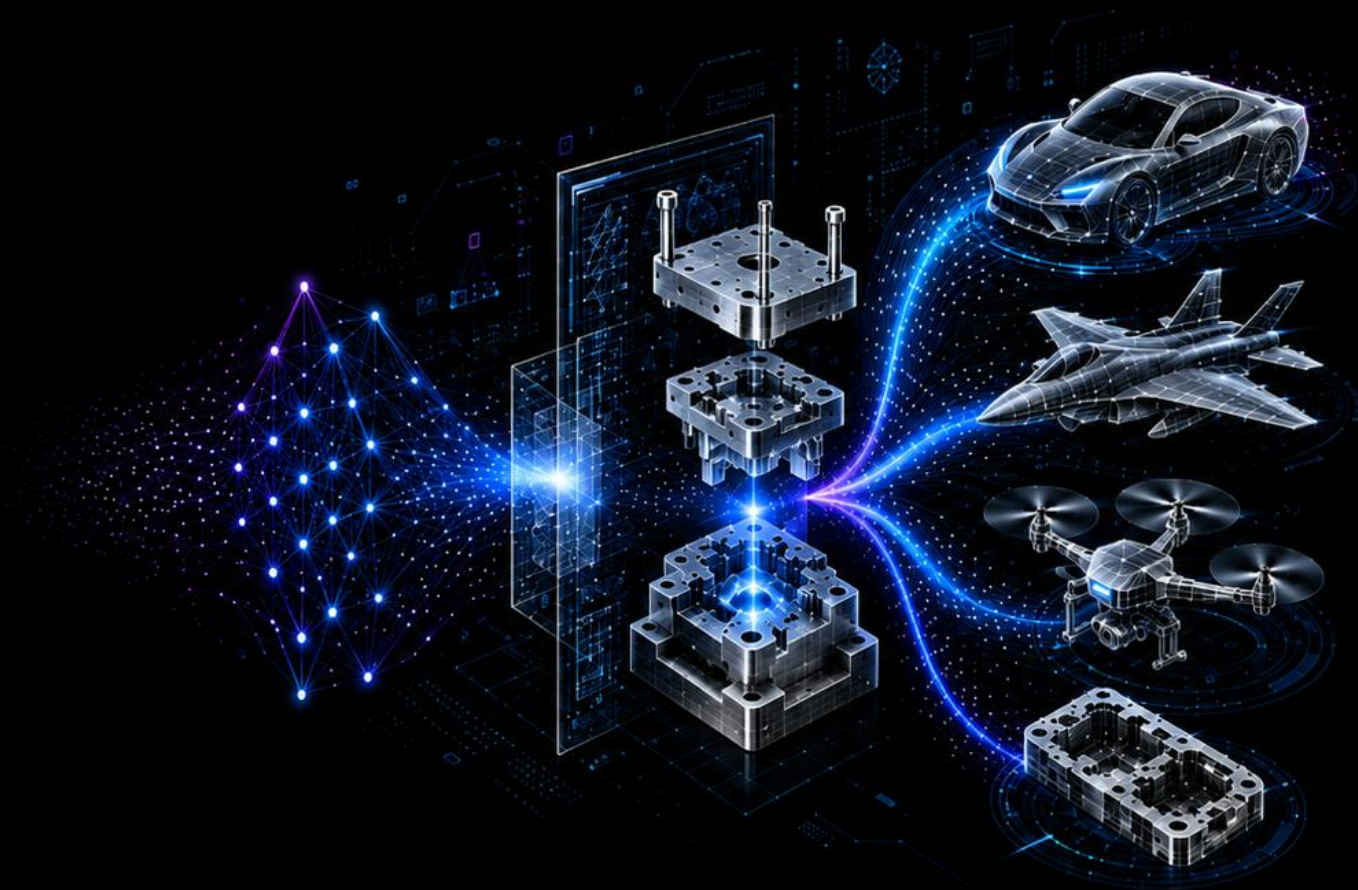
정규 스터디

8장 (성능 최적화)

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design
for Engineering
Applications Lab

1. 데이터를 사용한 성능 최적화

방법

✓ 방대한 데이터 수집

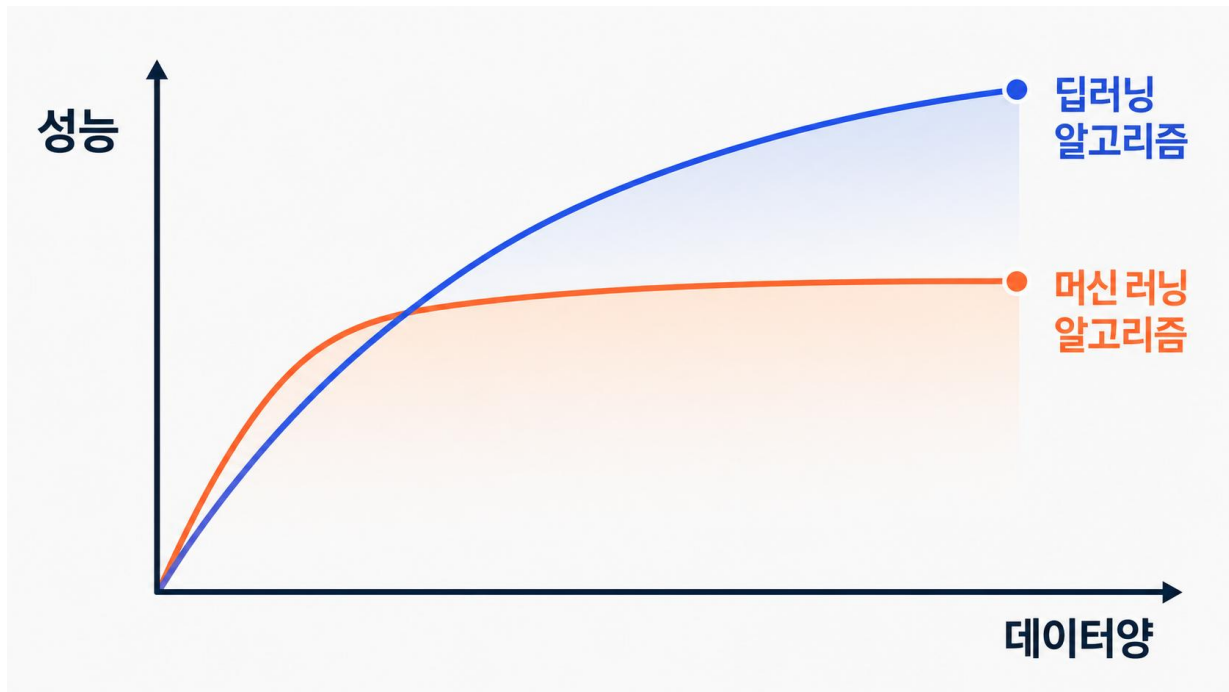
⇒ 오른쪽 그림과 같이 딥러닝과 머신러닝은 데이터 양이 많을 수록 성능이 좋아지는 것을 볼 수 있다.

✓ 데이터 생성

⇒ 데이터 수집의 한계가 있다면, 앞 5장에서 배운 내용과 같이 거대한 양의 데이터를 미리 학습한 “전이 학습”을 통해 해결할 수 있다

✓ 데이터 범위 조정

⇒ 활성화 함수를 이용하여 데이터셋의 범위를 조절하는 것 또한 성능 향상에 도움이 된다



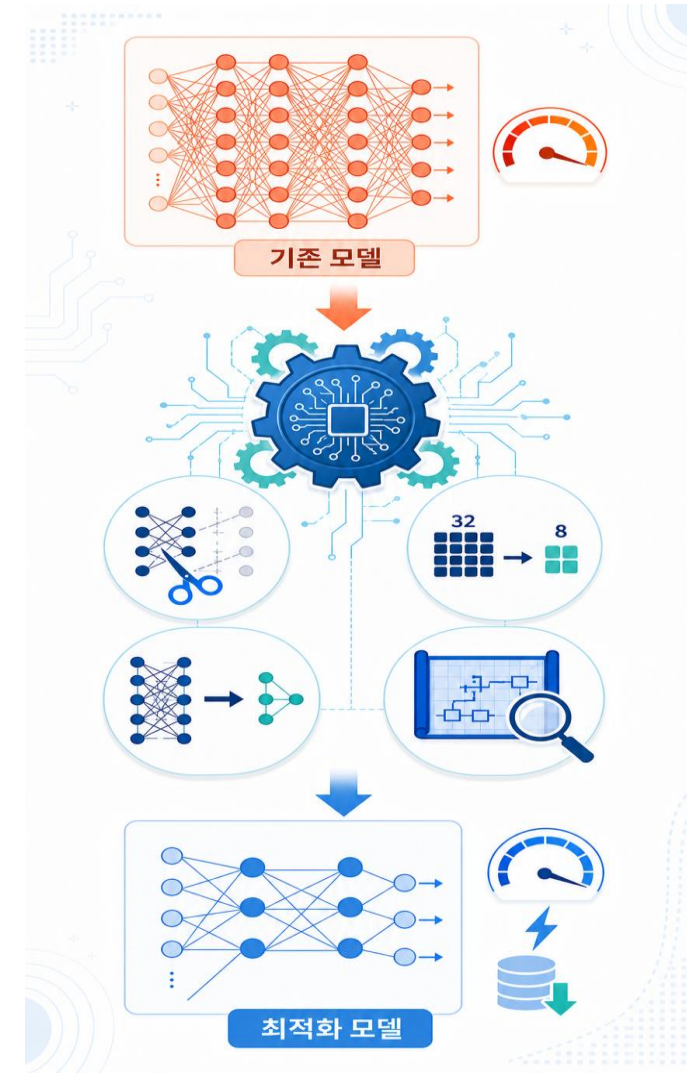
2. 알고리즘을 이용한 성능 최적화

방법

- ✓ 딥러닝과 머신러닝에 적용되는 알고리즘은 상당히 많이 존재한다. 그에 따라, 용도에 맞는 여러 알고리즘들도 다양하게 존재하기 때문에 적절한 알고리즘을 판단 하는 것이 이상적
- ✓ 또한, 적절한 알고리즘을 찾았다면 최고의 성능을 도출하기 위한 하이퍼파라미터 조정도 필요하다

하이퍼 파라미터

- 진단: 원인 분석
- 가중치: 초기 작은 난수 사용 or 사전 훈련 가중치 사용
- 학습률: 학습률을 조절하여 오차가 가장 작도록 이동
- 활성화 함수: 손실 함수와도 관련있기 때문에 신중한 선택 필요
- 배치 & 에폭: 배치 크기와 에폭 수
- 옵티마이저 & 손실 함수: 어떤 옵티마이저와 손실 함수가 최적인가
- 네트워크 구성: 최적의 네트워크 구성을 찾아야 한다

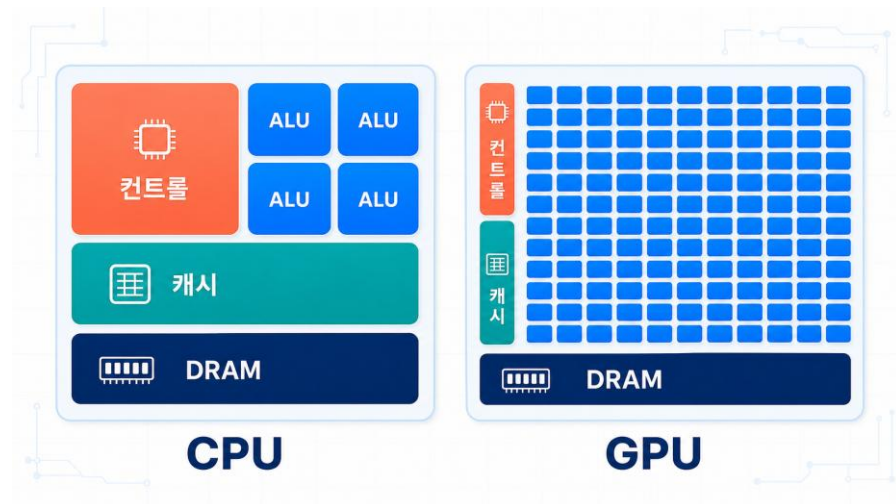
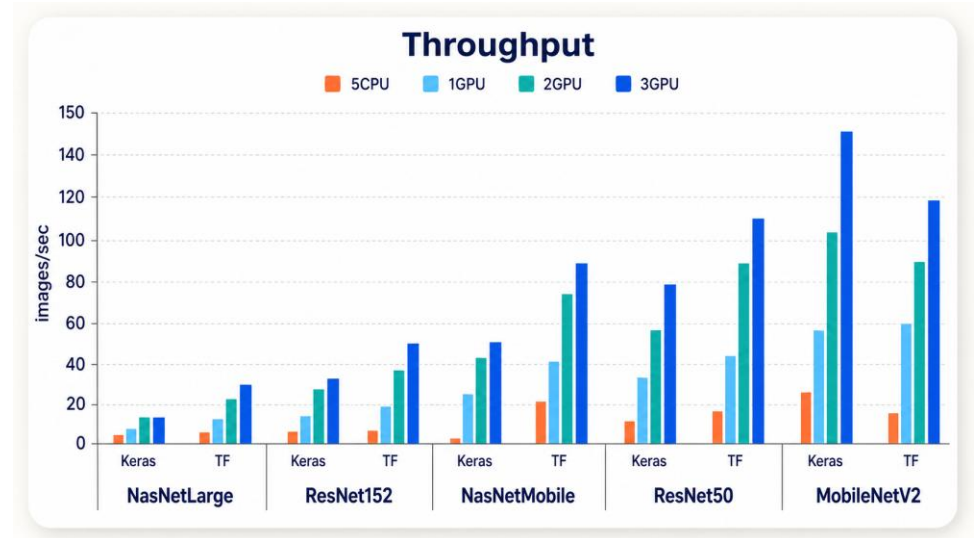


3. 하드웨어를 이용한 성능 최적화

CPU 와 GPU

- ✓ CPU: 직렬과 순차적인 연산에 강함
- ✓ GPU: 병렬적이고 동시적인 연산에 강함

⇒ 딥러닝에서는 역전파의 복잡한 미적분과 같이 병렬 연산을 해야 할 경우가 많고, 데이터를 수백~수천 개를 다뤄야 하는 상황이 많기 때문에 CPU보다는 GPU를 사용하는 것이 필수이다



4. 하이퍼 파라미터를 이용한 성능 최적화 - ① 배치 최적화

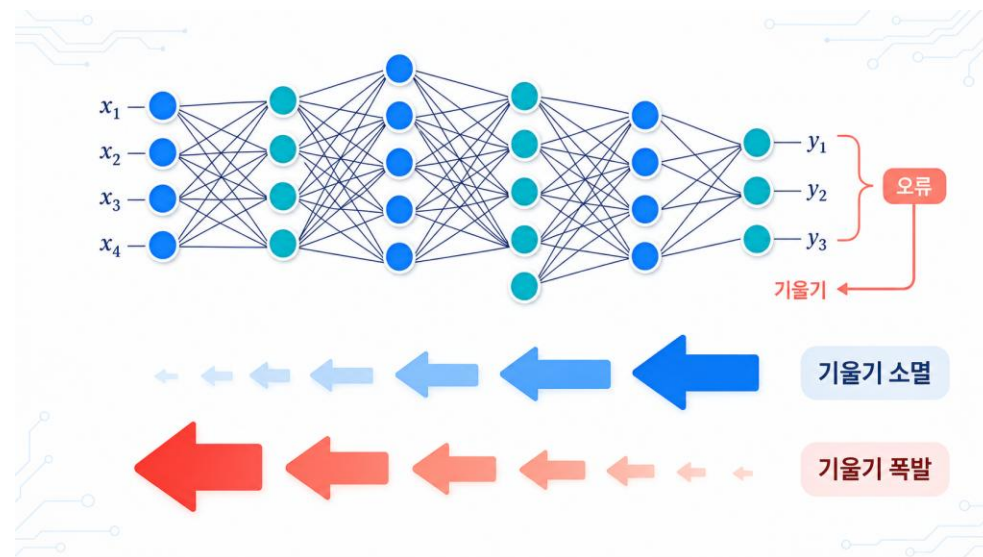
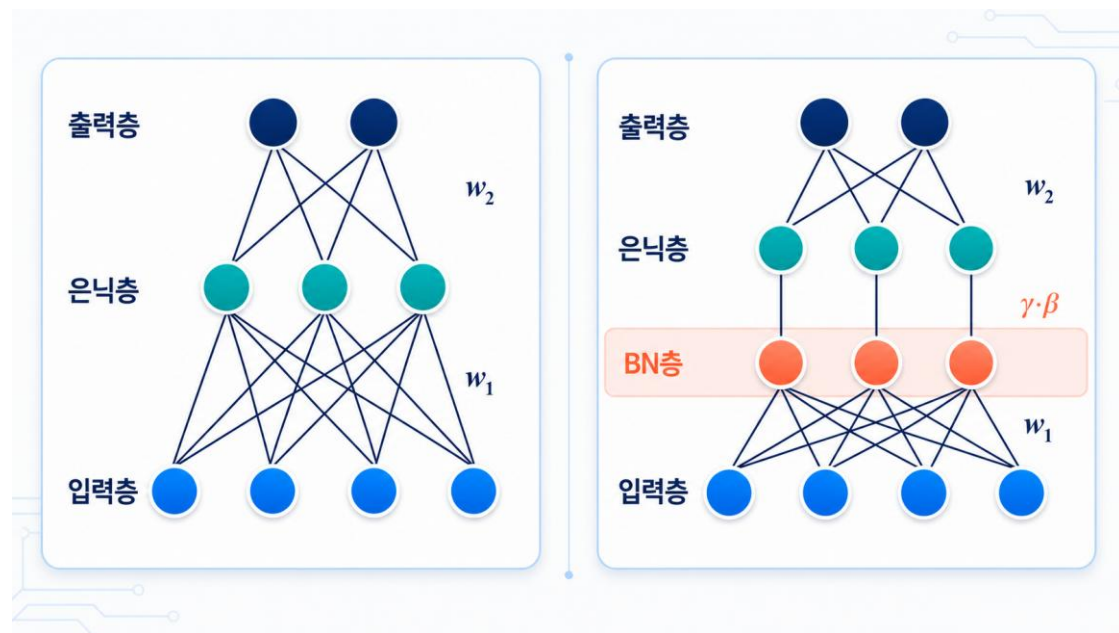
✓ 의미: 기울기 소멸이나 기울기 폭발과 같은 문제를 해결하기 위해, 데이터 분포를 안정시키는 방법이다.

✓ 기울기 소멸이나, 기울기 폭발이 일어나는 이유:

네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌게 되어 발생

✓ 해결 방법: 분산된 분포를 정규분포와 같이 표준화 하기 위해

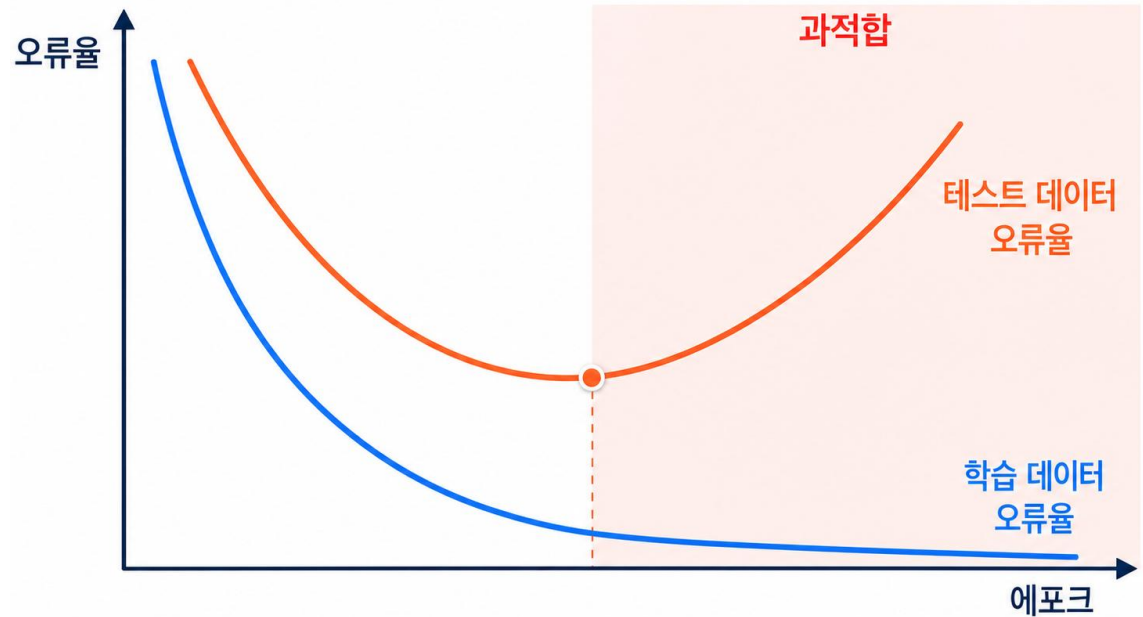
“미니 배치” 방식을 적용하여 평균은 0, 표준 편차는 1로 유지하도록 한다.



4. 하이퍼 파라미터를 이용한 성능 최적화 - ② 드롭 아웃을 이용한 최적화

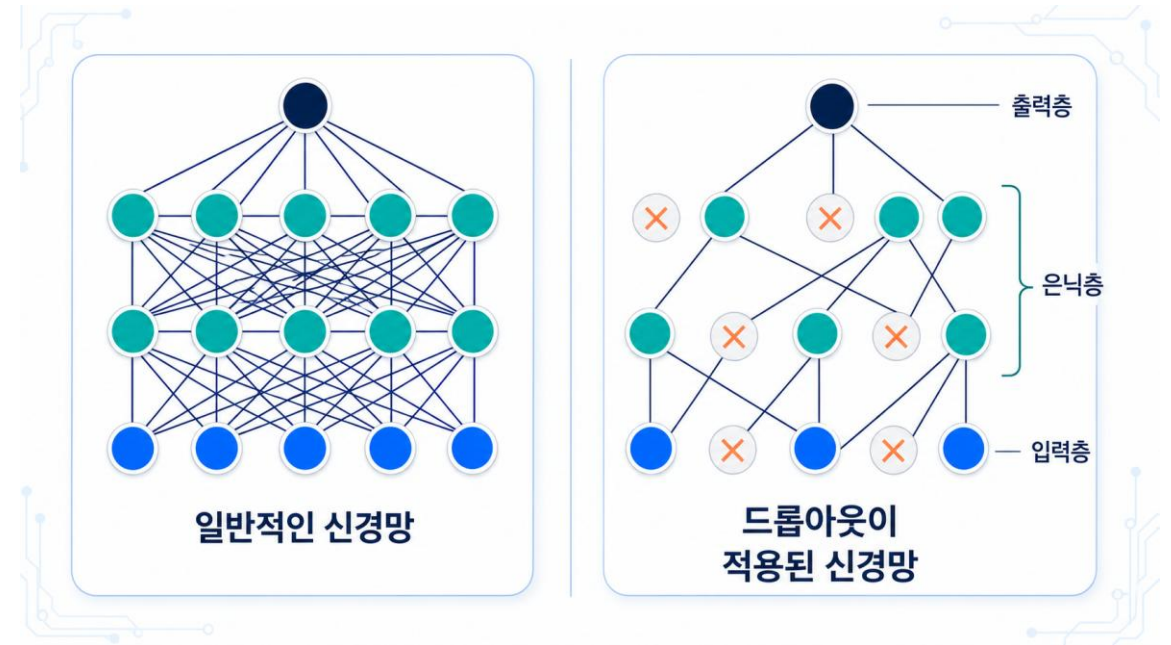
드롭 아웃의 사용 이유

- ✓ 훈련 데이터셋을 학습하다 보면,
데이터셋을 과하게 학습하게 되어 “과적합” 문제가 발생하게 된다.



사용 방법

- ✓ 드롭 아웃은 노드를 임의로 끄면서 학습하는 방법으로,
꺼진 노드는 신호가 전달 되지 않고,
켜져 있는 노드는 (드롭 아웃 비율)만큼 곱해져 성능을 평가하게 된다.



■ 배치 정규화와 드롭 아웃의 예제 코드: FashionMNIST 데이터를 이용한 정확도 평가

배치 정규화 적용 X

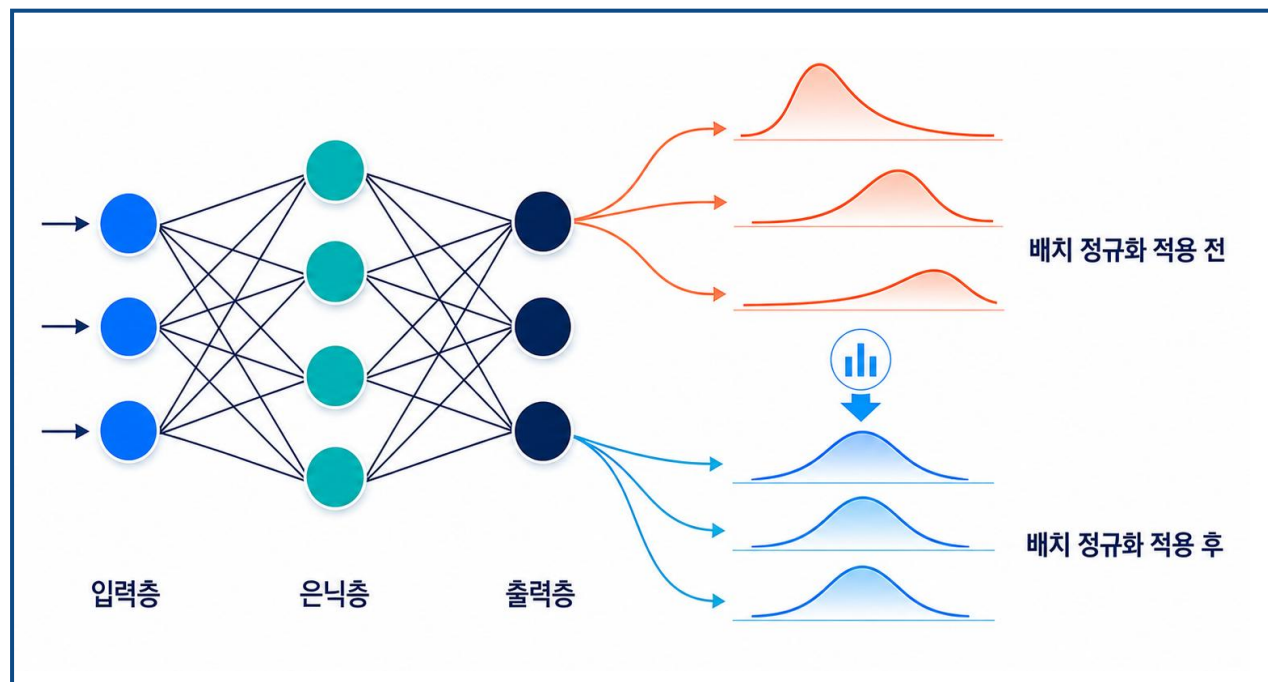
```
class NormalNet(nn.Module):
    def __init__(self):
        super(NormalNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48), # 28 x 28 = 784
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.ReLU(),
            nn.Linear(24, 10)
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

배치 정규화 적용 O

```
class BNNet(nn.Module):
    def __init__(self):
        super(BNNet, self).__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48),
            nn.BatchNorm1d(48),
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.BatchNorm1d(24),
            nn.ReLU(),
            nn.Linear(24, 10)
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

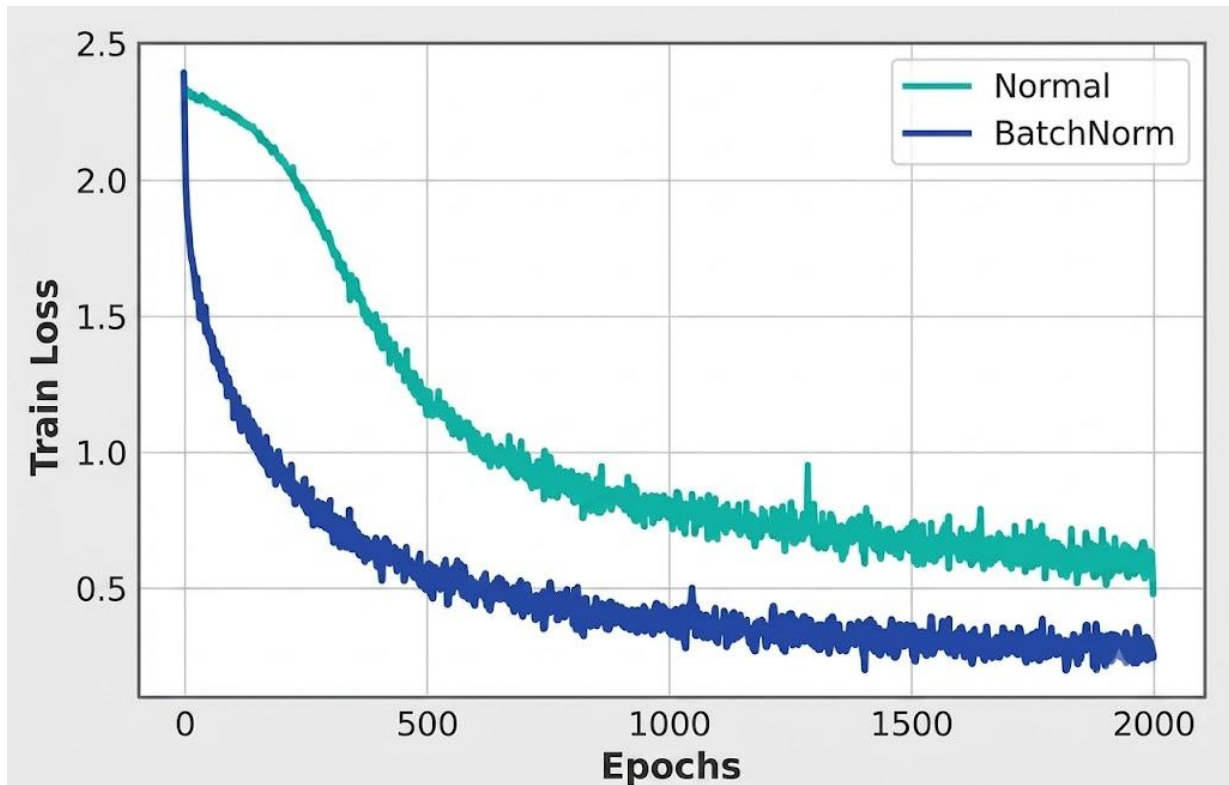


■ 배치 정규화와 드롭 아웃의 예제 코드

결과 비교

오른쪽 그림에서 보이는 것과 같이, 정규화가 적용된 것과 적용되지 않은 것 모두 오차가 줄어들고 있다.

하지만,
배치 정규화가 된 모델은 에폭이 진행될 수록 오차도 줄어들면서 안정적인 학습을 하고 있는 것이 보인다.



■ 배치 정규화와 드롭 아웃의 예제 코드

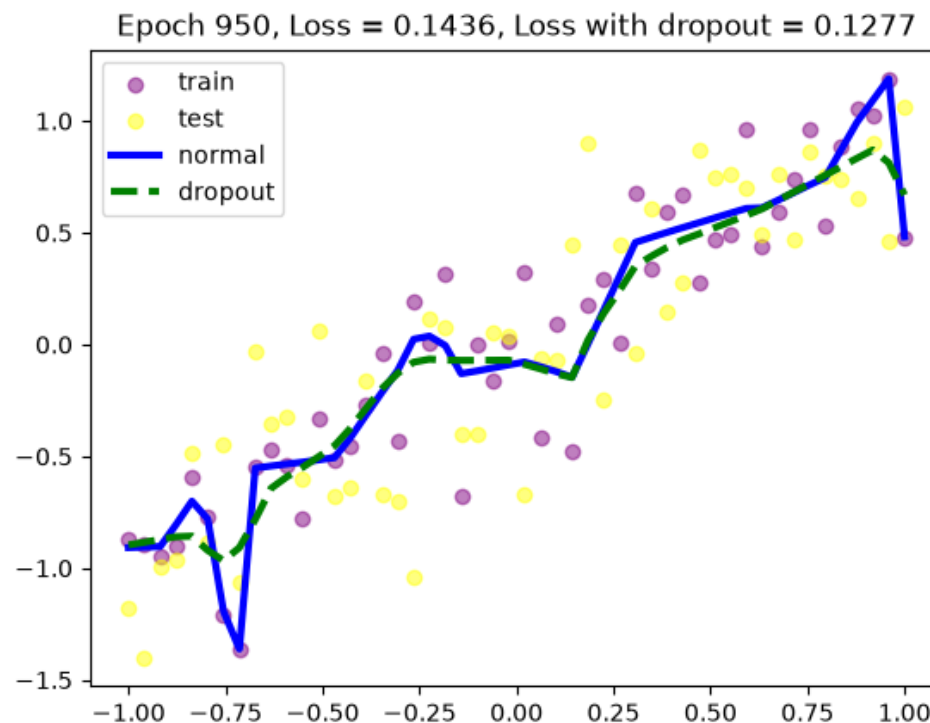
코드 설정

```
N_h = 100
model = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, 1),
)
..... 드롭 아웃이 적용 x

model_dropout = torch.nn.Sequential(
    torch.nn.Linear(1, N_h),
    torch.nn.Dropout(0.2),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, N_h),
    torch.nn.Dropout(0.2),
    torch.nn.ReLU(),
    torch.nn.Linear(N_h, 1),
)
..... 드롭 아웃이 적용 o
```

결과 비교

마지막 20번째 에폭에서 보이는거 처럼, 훈련 횟수가 늘어날 수록
드롭 아웃이 적용되지 않은 모델(파란선)은 가장자리 자주색 점들까지 찾아간다.
이를 보아 과적합 현상을 보이고 있다고 판단할 수 있다.



■ 4. 하이퍼 파라미터를 이용한 성능 최적화 - ③ 조기 종료를 이용한 최적화

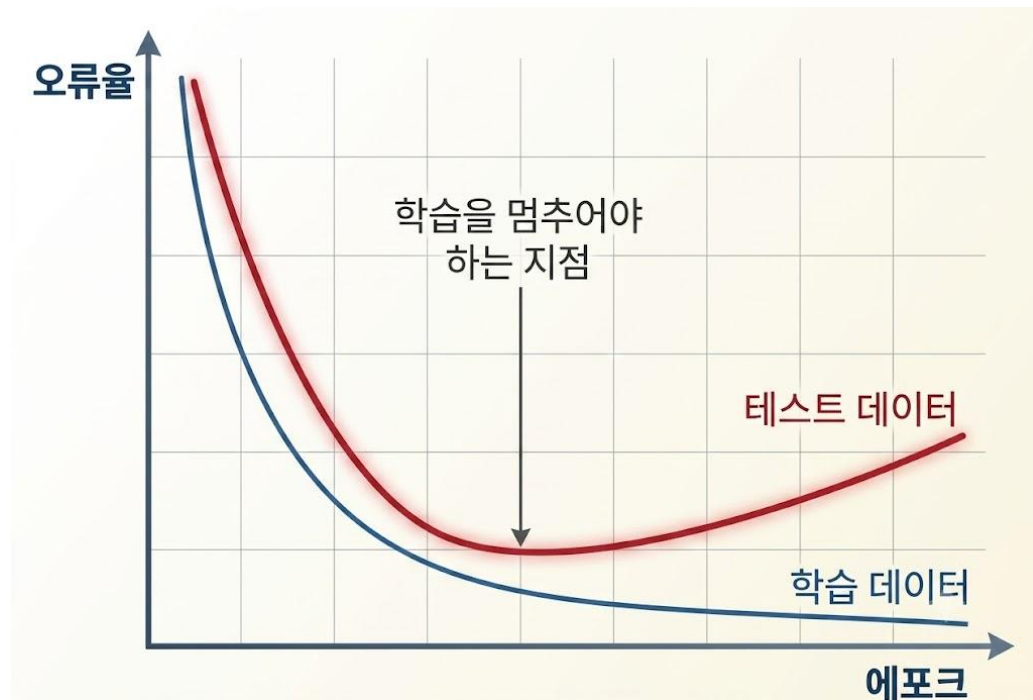
조기 종료의 사용 이유

- ✓ 사진에서 보이는 것과 같이 과적합이 발생하기 전에 학습을 멈춘다면 더욱 더 효율적으로 성능 향상 가능

사용 방법

해당 예제에서는 조기 종료 뿐만 아니라 학습률 까지 조정하여 성능 향상을 보여준다.

- ✓ 학습률 조정: 스케줄러에서 설정한 patience 동안 오차 개선이 없다면 factor 만큼 학습률을 조정
- ✓ 조기 종료: 조기 종료 모델에서 설정한 patience 동안 오차 개선이 없다면 학습이 종료된다.



■ 조기 종료와 학습률 조정의 예제 코드: 핫도그 사진 판별 - ① 조기 종료만 실행

조건

[스케줄러]

```
class LRScheduler():  
    def __init__(  
        self, optimizer, patience=5, min_lr=1e-6, factor=0.5
```

[조기 종료]

```
class EarlyStopping():  
    def __init__(self, patience=5, verbose=False, delta=0, path=None):
```

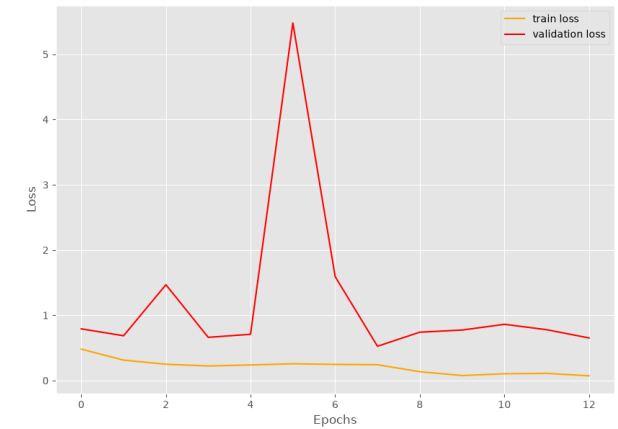
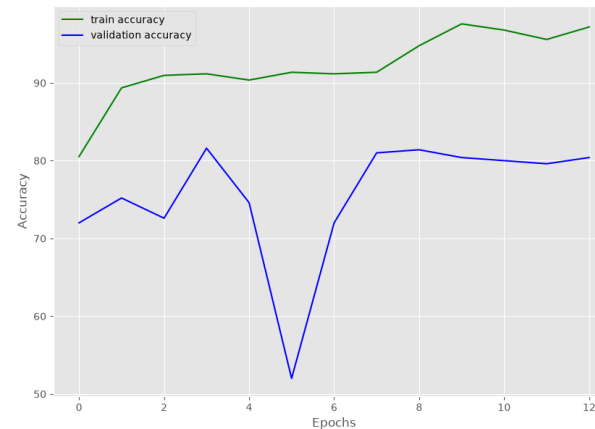
[학습에 사용할 기능 스위치]

```
args = {  
    'lr_scheduler': False,  
    'early_stopping': True,  
}
```

⇒ 스케줄러와 조기 종료의 patience는 기존 교재와 동일하고
조기 종료만 실행하였다.

결과

학습 과정에서 학습률이 너무 커서 중간에 정확도가 확 떨어지지만,
100 단계의 에폭 중 16 단계까지의 과정을 거쳐 오차 정확도 79.6%가 나왔다



■ 조기 종료와 학습률 조정의 예제 코드 - ② 스케줄러 + 조기 종료

조건

[스케줄러]

```
class LRScheduler():  
    def __init__(  
        self, optimizer, patience=5, min_lr=1e-6, factor=0.5
```

[조기 종료]

```
class EarlyStopping():  
    def __init__(self, patience=10, verbose=False, delta=0, path=None):
```

[학습에 사용할 기능 스위치]

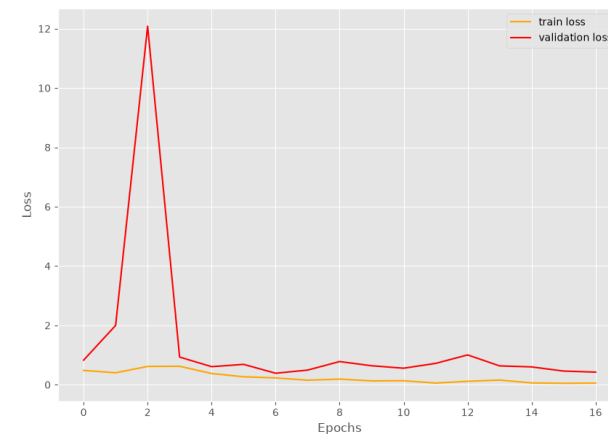
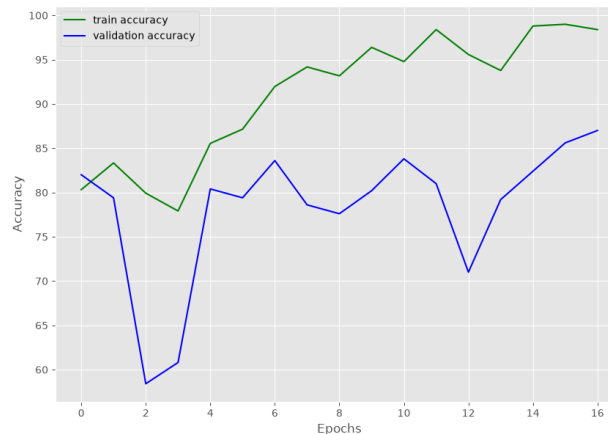
```
args = {  
    'lr_scheduler': True,  
    'early_stopping': True,  
}
```

⇒ 스케줄러와 조기 종료의 patience는 각각 임의로 5 와 10으로 설정
했고, 스케줄러와 조기종료 모두를 실행하였다

결과

스케줄러와 조기 종료를 함께 사용하면 성능이 정체되었을 때, 바로 학습을
종료하는 것이 아닌 학습률을 낮춰 다시 실행하기 때문에 성능이 향상될 수 있다.

아래 사진에서 보이는 것과 같이 조기 종료만 실행했을 때보다
최종 오차 정확도 85.6% 로 더 높아진 것을 볼 수 있다.



■ 조기 종료와 학습률 조정의 예제 코드 - ③ 스케줄러 + 조기 종료 (patience 변경 버전)

조건

[스케줄러]

```
class LRScheduler():  
    def __init__(  
        self, optimizer, patience=2, min_lr=1e-6, factor=0.5
```

[조기 종료]

```
class EarlyStopping():  
    def __init__(self, patience=8, verbose=False, delta=0, path=None):
```

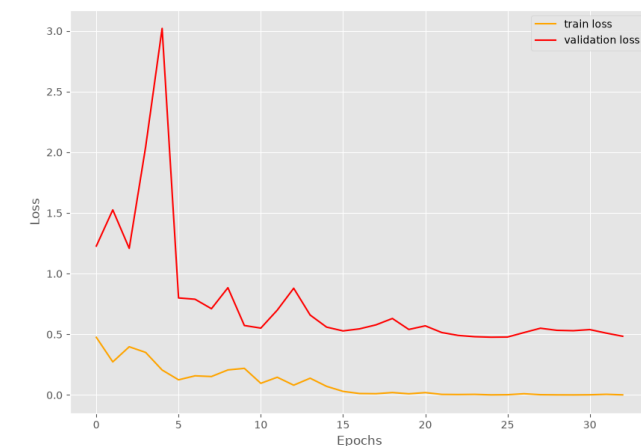
[학습에 사용할 기능 스위치]

```
args = {  
    'lr_scheduler': True,  
    'early_stopping': True,  
}
```

⇒ 기존 조건에서 patience만 크게 차이가 나도록 하여 실행함.

결과

바로 전 가정과 비교했을 때, 학습 시간은 오래 걸렸지만
에폭도 약 2배로 진행되었고, 최종 오차 정확도 또한 87.2%로 증가하게 되었다.



■ 조기 종료와 학습률 조정의 예제 코드 - ④ 스케줄러와 조기 종료 모두 끄기

조건

[스케줄러]

```
class LRScheduler():  
    def __init__(  
        self, optimizer, patience=5, min_lr=1e-6, factor=0.5
```

[조기 종료]

```
class EarlyStopping():  
    def __init__(self, patience=5, verbose=False, delta=0, path=None):
```

[학습에 사용할 기능 스위치]

```
args = {  
    'lr_scheduler': False,  
    'early_stopping': False,  
}
```

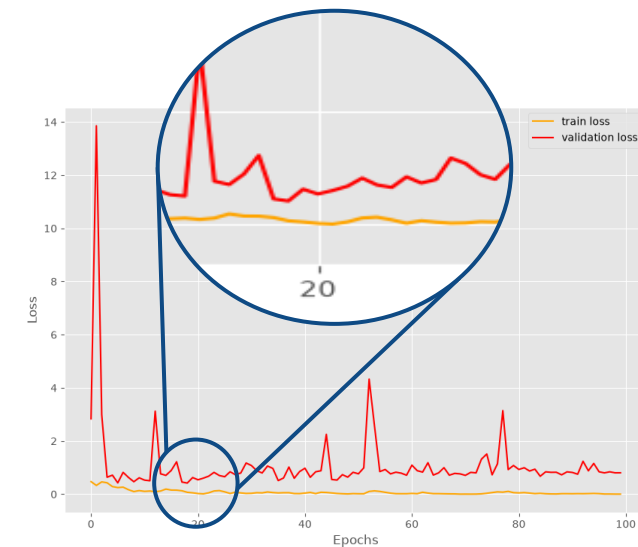
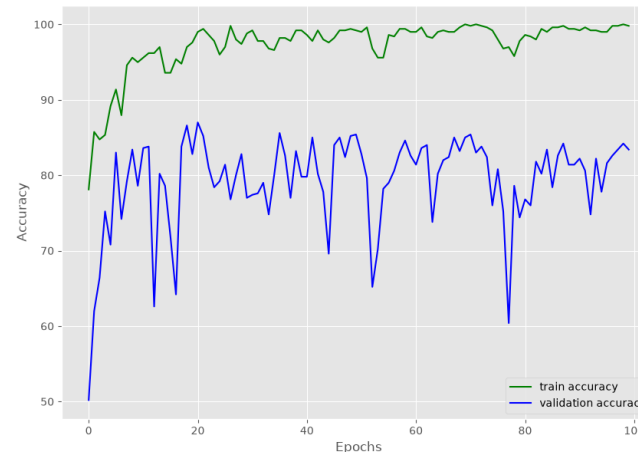
⇒ 기존 교재 설정들을 유지하고 모든 기능을 해제

결과

모든 기능을 쓰지 않았기 때문에, 학습 시간이 22분으로 굉장히 긴 시간 학습이 진행되었다.

특히 에폭 20에서는 미세하게 우상향 하는 것을 보아 과적합이 시작 되었음을 알 수 있다.

⇒ 이러한 가정들을 통해서 학습률 조정과 조기 종료가 얼마나 성능을 향상시키는지 확인할 수 있다.



Any Questions?

이름 최준석

소속 한양대학교 기계공학과

이메일 chw10109@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

